Combinatorics: nCr, Factorials & Pascal's Triangle

A comprehensive guide to combinatorial mathematics — from foundational theory to production-grade implementation. Covering binomial coefficients, factorial computation, and Pascal's triangle with real-world applications in probability, machine learning, and algorithm design.

DEVELOPED BY TALENCIAGLOBAL | DISCRETE MATHEMATICS | ALGORITHM DESIGN

Topic Classification

Domain Overview

Category: Combinatorics / Discrete Mathematics / Algorithm Design

Domain: Mathematics → Combinatorics → Binomial Coefficients

Difficulty: Beginner to Intermediate

Prerequisites: Basic arithmetic, loops, recursion basics, understanding of multiplication and division

Industry Relevance

Probability & Statistics

Core engine behind binomial distributions and hypothesis testing

Bioinformatics

Sequence counting and genotype enumeration

Machine Learning

Binomial distributions, feature combinations, model selection

Cryptography

Key space enumeration and combinatorial security analysis

What Is Combinatorics?

Combinatorics is the branch of mathematics concerned with counting, arrangement, and combination of objects. Three foundational constructs underpin the entire field:

Factorial (n!)

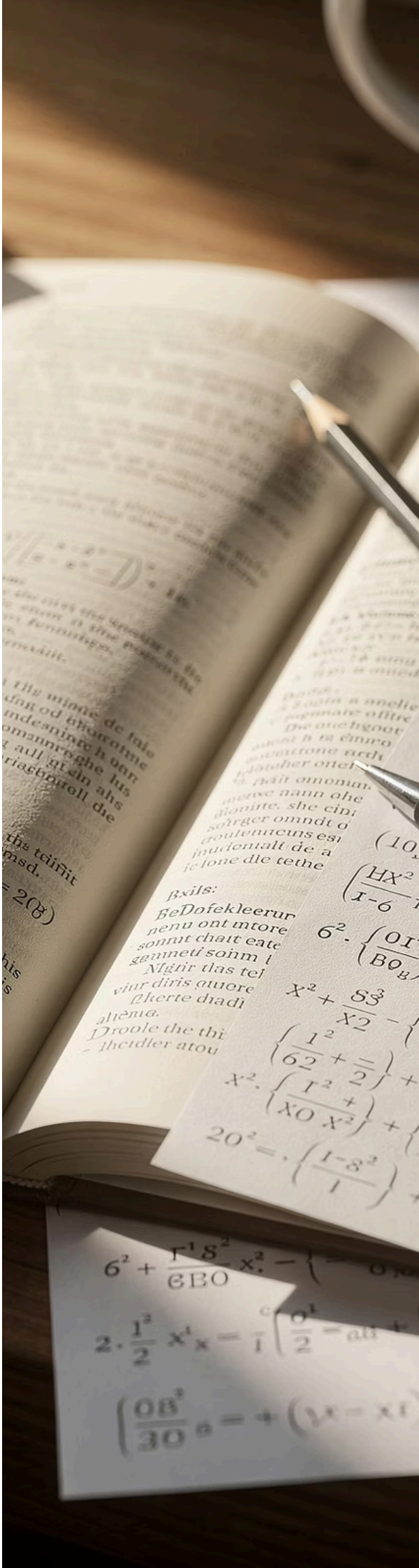
The product of all positive integers up to n. Formally: **n! = n × (n−1) × … × 2 × 1**, with the special case **0! = 1**. Example: 5! = 120.

nCr — Binomial Coefficient

The number of ways to choose r unordered items from n distinct items. Formula: **C(n,r) = n! / (r! × (n−r)!)**, valid for 0 ≤ r ≤ n.

Pascal's Triangle

A triangular array where each entry equals the sum of the two directly above it. Row n (0-indexed) yields all binomial coefficients C(n,0), C(n,1), …, C(n,n).



Core Components at a Glance

Each of the three building blocks has a distinct operational identity, recurrence relation, and base case:

Component	Factorial	nCr	Pascal's Triangle
Core Operation	Multiplication	Division of factorials	Addition of two numbers
Recurrence	$n! = n \times (n-1)!$	$C(n,r) = C(n-1,r-1) + C(n-1,r)$	Same as nCr recurrence
Base Case	$0! = 1$	$C(n,0) = C(n,n) = 1$	Top entry = 1
Primary Risk	Overflow for $n \geq 21$ (64-bit)	Intermediate overflow	$O(n^2)$ memory for full table

ⓘ Statistical anecdote: The number 52! (arrangements of a deck of cards) is approximately 8×10^{67} — larger than the estimated number of atoms in the observable universe (~10^{80} is close, but $52! \approx 8 \times 10^{67}$). Every time a deck is shuffled, the resulting order has almost certainly never existed before in human history.

Why Does Combinatorics Exist?

Problems It Solves

Counting Without Enumeration

Avoids exponential blow-up when listing all subsets is computationally infeasible.

Probability Computation

Enables exact odds for gambling, risk analysis, and random sampling without simulation.

Polynomial Expansion

The Binomial Theorem expands $(x+y)^n$ in closed form using binomial coefficients.

What Happens Without It

Lottery Odds

To compute "choose 5 from 50," you would need to enumerate all 2,118,760 subsets — impossible by hand.

Binomial Expansion

Expanding $(x+y)^{10}$ by distributing manually produces 1,024 intermediate terms.

Regulatory Disclosure

Lotteries would have no mathematically defensible way to publish jackpot odds.

Real-World Motivation

Lottery Probability

Powerball requires choosing 5 numbers from 69. The total combinations: **C(69,5) = 11,238,513**. Without combinatorics, regulators cannot publish legally defensible jackpot odds. The probability of winning is approximately 1 in 292 million when the Powerball number is included.

Machine Learning

The binomial distribution models the number of successes in n independent Bernoulli trials — directly powering click-through rate models, A/B testing frameworks, and quality control systems. Every logistic regression implicitly relies on combinatorial probability theory.

Genetics

The number of possible genotypes arising from parental allele combinations is given by binomial coefficients. For a diploid organism with n heterozygous loci, the number of distinct offspring genotypes is C(2n, n) — a direct application of Pascal's triangle.

Evolution & History of Combinatorics

1

~1000 CE

Indian Mathematicians — Bhaskaracharya described "Meru prastāra," the earliest known form of Pascal's triangle in Sanskrit mathematical texts.

2

~1100 CE

Jia Xian & Yang Hui — Chinese mathematician Jia Xian introduced the triangle; Yang Hui popularised it, giving rise to the name "Yang Hui's Triangle" in East Asia.

3

1653

Blaise Pascal — Published *Traité du triangle arithmétique*, formalising the triangle's properties and establishing the foundation for modern combinatorics.

4

1665

Isaac Newton — Generalised the binomial theorem to non-integer exponents, extending combinatorics far beyond discrete counting into analysis.

5

18th Century

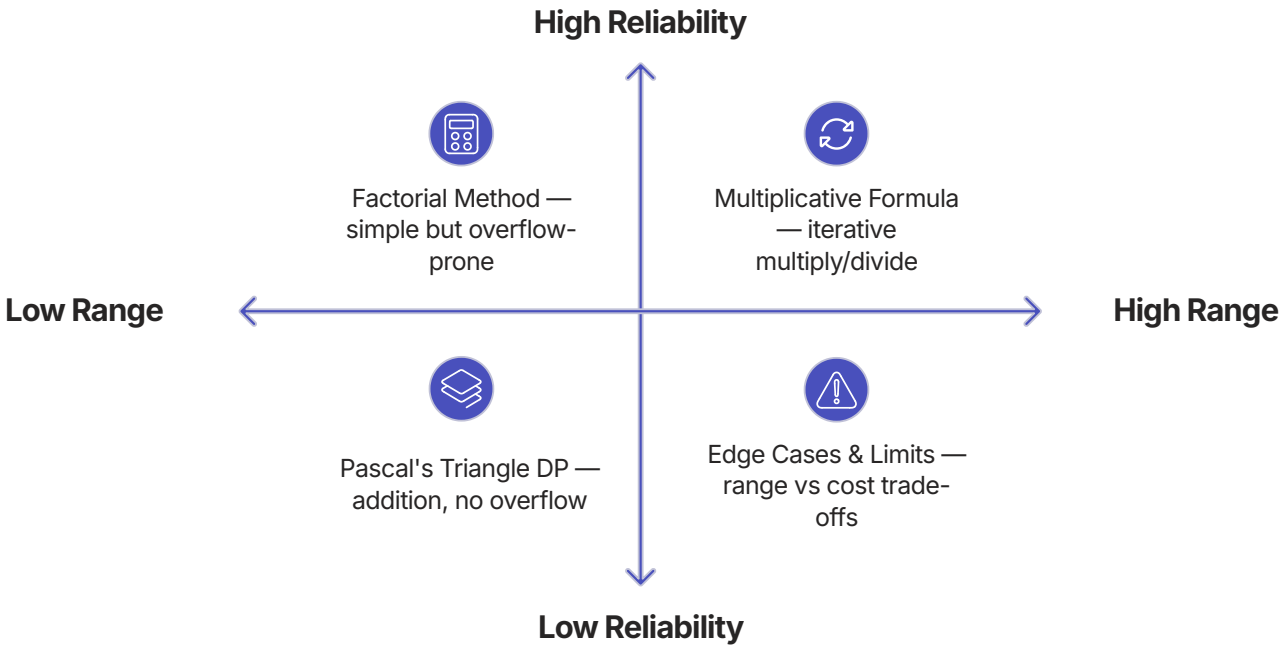
Christian Kramp — Introduced the factorial notation n!, standardising mathematical communication across Europe.

6

20th Century–Present

Dynamic Programming & Built-ins — Pascal's triangle DP became a computational staple. Python 3.8+ introduced `math.comb`; Lucas theorem enables modular combinatorics for competitive programming.

How It Works: Three Approaches to nCr



Each method represents a different trade-off between simplicity, range, and computational cost. The factorial method is intuitive but fails for n ≥ 21 in 64-bit arithmetic. The multiplicative formula is optimal for single queries. Pascal's DP excels when many coefficients for the same n are needed.

Pascal's Triangle: Structure & Patterns

Rows 0 Through 5

```
Row 0:      1
Row 1:     1 1
Row 2:    1 2 1
Row 3:   1 3 3 1
Row 4:  1 4 6 4 1
Row 5: 1 5 10 10 5 1
```

Each entry = sum of the two directly above it. Example: 10 (row 5, position 2) = 4 + 6.

Key Properties

- **Row Sum = 2^n**

  The sum of all entries in row n equals 2^n. Row 5 sums to 1+5+10+10+5+1 = 32 = 2^5.

- **Symmetry**

  C(n,r) = C(n, n−r). Each row reads the same forwards and backwards.

- **Hockey Stick Identity**

  The diagonal sum identity: Σ C(i,r) for i=r to n equals C(n+1, r+1).

- **Sierpiński Fractal**

  Marking odd entries (mod 2) in Pascal's triangle produces the Sierpiński triangle — a self-similar fractal.

Step-by-Step: Multiplicative Computation of C(7,3)

The multiplicative formula avoids computing large factorials by interleaving multiplication and division. Below is the exact computation of C(7,3) = 35:

01

Initialise

Set `result = 1`. We will iterate i from 1 to r=3, using the formula: `result = result × (n − r + i) // i`

02

Step i=1

`result = 1 × (7 − 3 + 1) = 1 × 5 = 5` → divide by 1 → **result = 5**

03

Step i=2

`result = 5 × (7 − 3 + 2) = 5 × 6 = 30` → divide by 2 → **result = 15**

04

Step i=3

`result = 15 × (7 − 3 + 3) = 15 × 7 = 105` → divide by 3 → **result = 35**

05

Final Answer

C(7,3) = 35. Verified: 7! / (3! × 4!) = 5040 / (6 × 24) = 5040 / 144 = 35. ✓

Key insight: At each step i, the running product is always exactly divisible by i. This guarantees integer arithmetic throughout — no floating-point rounding errors.

Types & Variants

Factorial Variations

Type	Definition	Use Case
Double factorial (n!!)	Product of every second number	Symmetric group permutations
Subfactorial (!n)	Number of derangements	Combinatorial derangements
Primorial (n#)	Product of primes ≤ n	Number theory

Pascal's Triangle Variants

Type	Modification	Use
Sierpiński (mod 2)	Mark odd entries	Fractal visualisation
Lucas triangle	Mod p version	Generalised binomials
Leibniz harmonic	Reciprocals of entries	Harmonic number connections

Binomial Coefficient Variants

Variant	Formula	Use Case
Standard nCr	$n!/(r!(n-r)!)$	Basic combinations
Multinomial	$n!/(r_1!r_2!...r_k!)$	Partition into multiple groups
Modulo prime	Lucas theorem	Competitive programming
Central binomial	C(2n,n)	Catalan numbers

> The central binomial coefficient C(2n,n) appears in the Catalan number formula $C_n = C(2n,n)/(n+1)$, which counts the number of valid parenthesisation sequences — critical in compiler design and expression parsing.

Computation Method Trade-offs

Selecting the right algorithm depends on the scale of n, the number of queries, and whether exact integer results are required:

Method	Time	Space	Overflow Risk	Best Use Case
Factorial (direct)	O(n)	O(1)	Very High	Only tiny n (≤ 12)
Multiplicative (stepwise)	O(r)	O(1)	Moderate	$n \leq 10^{12}$, r small
Pascal DP (2D table)	$O(n^2)$	$O(n^2)$	Low	Small n (≤ 5000), many queries
Pascal DP (1D array)	$O(n^2)$	O(n)	Low	Small n, memory constrained
Precomputed + mod inverse	O(n) pre, O(1) query	O(n)	None (modulo)	Modular combinatorics
Log gamma approximation	O(1)	O(1)	Float error	Approximate, very large n

Real-World Use Case 1: Lottery Odds Calculator

Business Problem

Compute the probability of winning a lottery where a player selects 6 numbers from 49. Regulatory bodies require accurate integer odds for public disclosure — floating-point approximations are legally insufficient.

Solution Approach

Apply the multiplicative formula for C(49,6) using 64-bit integers, interleaving multiplication and division to prevent overflow:

$$C(49,6) = (49 \times 48 \times 47 \times 46 \times 45 \times 44) / 720 = 13{,}983{,}816$$

Odds of winning: approximately **1 in 13,983,816**. The multiplicative method ensures exact integer arithmetic without computing 49! directly.

Key Lessons

Interleave Operations

Multiply then divide at each step — never compute the full numerator first.

64-bit Safety

C(66,33) ≈ 7.2×10^{18} is the last value fitting in an unsigned 64-bit integer. C(67,33) overflows.

Exact Integer Required

Regulatory disclosure demands exact values — log-gamma approximations are inappropriate here.

Real-World Use Case 2: Binomial Probability in A/B Testing

Business Problem

An e-commerce platform observes 45 conversions out of 1,000 visitors in variant B. The null hypothesis assumes a conversion rate of 4%. The team must compute the p-value: the probability of observing at least 45 successes under the null.

Solution

Compute the binomial CDF tail sum:

$$\sum_{k=45}^{1000} \binom{1000}{k} (0.04)^k (0.96)^{1000-k}$$

Each binomial coefficient is computed using the multiplicative method in log space to avoid underflow. Precomputed log-factorials accelerate the summation.

Architecture

01

Precompute Log Factorials

Build an array `log_fact[i] = log(i!)` for i = 0 to 1000 in O(n) time.

02

Compute Log of Each Term

For each k: `log_term = log_fact[1000] − log_fact[k] − log_fact[1000−k] + k×log(0.04) + (1000−k)×log(0.96)`

03

Exponentiate and Sum

Convert back from log space and accumulate the tail probability.

> Industry stat: A/B testing frameworks at major tech companies (Google, Meta, Amazon) process millions of such binomial p-value computations daily. Log-space arithmetic is non-negotiable at this scale.

Real-World Use Case 3: Grid Path Counting in Robotics

Business Problem

A logistics robot must navigate from the top-left to the bottom-right of a 20×20 grid, moving only right or down. How many distinct paths exist?

Combinatorial Insight

Any path consists of exactly 40 moves: 20 rights and 20 downs. The number of distinct paths equals the number of ways to choose which 20 of the 40 moves are "right":

$$C(40, 20) = 137{,}846{,}528{,}820$$

This exact value fits comfortably within a 64-bit integer.

Solution Approach

Compute using Pascal's triangle DP (1D array) — the grid is small enough that $O(n^2)$ is acceptable, and the approach builds DP intuition transferable to more complex routing problems.

Scaling Boundary

64-bit Safe

Large binomials fit into 64-bit integers up to approximately C(66,33). Beyond that, use Python big integers or modular arithmetic.

Production Note

For warehouse routing with grids larger than 33×33, switch to modular arithmetic or arbitrary-precision libraries.

Python Implementations

Example 1: Factorial — Iterative and Recursive

```python
def factorial_iter(n: int) -> int:
    """Iterative factorial. Returns 1 for n=0."""
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def factorial_rec(n: int) -> int:
    """Recursive factorial (not recommended for large n)."""
    return 1 if n <= 1 else n * factorial_rec(n - 1)

print(factorial_iter(5))  # 120
print(factorial_rec(5))   # 120
```

Example 2: nCr Using Multiplicative Method

```python
def ncr_multiplicative(n: int, r: int) -> int:
    """Return C(n, r) using multiplicative formula.
    Works for n up to ~60 with 64-bit; Python big ints handle larger."""
    if r < 0 or r > n:
        return 0
    r = min(r, n - r)  # exploit symmetry C(n,r) = C(n,n-r)
    result = 1
    for i in range(1, r + 1):
        result = result * (n - r + i) // i
    return result

print(ncr_multiplicative(10, 3))   # 120
print(ncr_multiplicative(52, 5))   # 2,598,960 (poker hands)
```

C(52,5) = 2,598,960 represents the total number of distinct 5-card poker hands from a standard 52-card deck — the foundation of all poker probability tables used in casinos worldwide.

Pascal's Triangle: Python Implementation

```python
def pascal_triangle_row(n: int) -> list[int]:
    """Return row n (0-indexed) of Pascal's triangle using 1D DP.
    Updates in reverse to avoid overwriting needed values."""
    row = [1] * (n + 1)
    for i in range(1, n + 1):
        for j in range(i - 1, 0, -1):
            row[j] = row[j] + row[j - 1]
    return row

def compute_all_binomials_upto(n_max: int) -> list[list[int]]:
    """Return entire Pascal triangle rows 0..n_max."""
    triangle = [[1]]
    for i in range(1, n_max + 1):
        prev = triangle[-1]
        curr = [1]
        for j in range(1, i):
            curr.append(prev[j-1] + prev[j])
        curr.append(1)
        triangle.append(curr)
    return triangle

row5 = pascal_triangle_row(5)
print(row5)  # [1, 5, 10, 10, 5, 1]
```

⚠ When updating a 1D Pascal row, always iterate in **reverse order** (from right to left). Forward iteration overwrites values still needed for the current update step, producing incorrect results — a common intermediate-level mistake.

Hands-On Labs

1

Lab 1: Compare nCr Computation Methods

Difficulty: Intermediate | **Objective:** Implement all three methods and compare output, speed, and range limits.

- Write `ncr_factorial(n,r)` using `math.factorial`
- Write `ncr_multiplicative(n,r)` as shown above
- Write `ncr_pascal(n,r)` using 1D DP table
- For n=30, r=15: verify all return **155,117,520**
- Test n=60, r=30 — identify which method overflows or is slow
- Time each method for n=1000, r=500 using `timeit`

Validation: All correct methods return 1.183×10^{299} for n=1000, r=500 (Python big integers handle this). **Learning outcome:** Multiplicative is best for single large nCr; Pascal DP is better for many queries with small n.

2

Lab 2: Pascal's Triangle Mod 2 — Sierpiński Fractal

Difficulty: Intermediate | **Objective:** Generate Pascal's triangle modulo 2 to visualise the Sierpiński triangle fractal.

- Generate the first 32 rows using 1D DP
- For each entry, compute `entry % 2`
- Print a grid: odd entries as `*`, even entries as spaces
- Observe the self-similar fractal pattern emerging

Learning outcome: Connect combinatorics with modular arithmetic and fractal geometry — a bridge between discrete mathematics and computer graphics.

Performance Considerations

Complexity Analysis

Operation	Time	Space	Note
n! (naïve)	O(n)	O(1)	Value has O(n log n) bits
C(n,r) multiplicative	O(r)	O(1)	Best for single query
Pascal DP (single row)	$O(n^2)$	O(n)	Good for many r, same n
Pascal DP (full triangle)	$O(n^2)$	$O(n^2)$	Overkill for single coefficient
Precomputed + mod inverse	O(n) pre, O(1)	O(n)	Modular arithmetic only

Key Overflow Boundaries

64-bit Unsigned Max ≈ 1.8×10^{19}

C(66,33) ≈ 7.2×10^{18} fits. C(67,33) overflows silently in C/C++/Java.

20! ≈ 2.4×10^{18}

Fits in 64-bit. 21! overflows. Use Python's arbitrary-precision integers for n ≥ 21.

Full Pascal Triangle n=10,000

Requires ~100 million integers ≈ 800 MB. Use 1D rolling array for memory-constrained environments.

Optimisation Techniques

Exploit Symmetry

Always compute `r = min(r, n−r)` before the loop. Halves the number of iterations.

Use math.comb (Python 3.8+)

C-optimised built-in. Benchmarks show 10–50× faster than pure Python multiplicative for large n.

Log Space for Probabilities

For large n in statistical computations, use `math.lgamma` to avoid underflow. Never use for exact integer results.

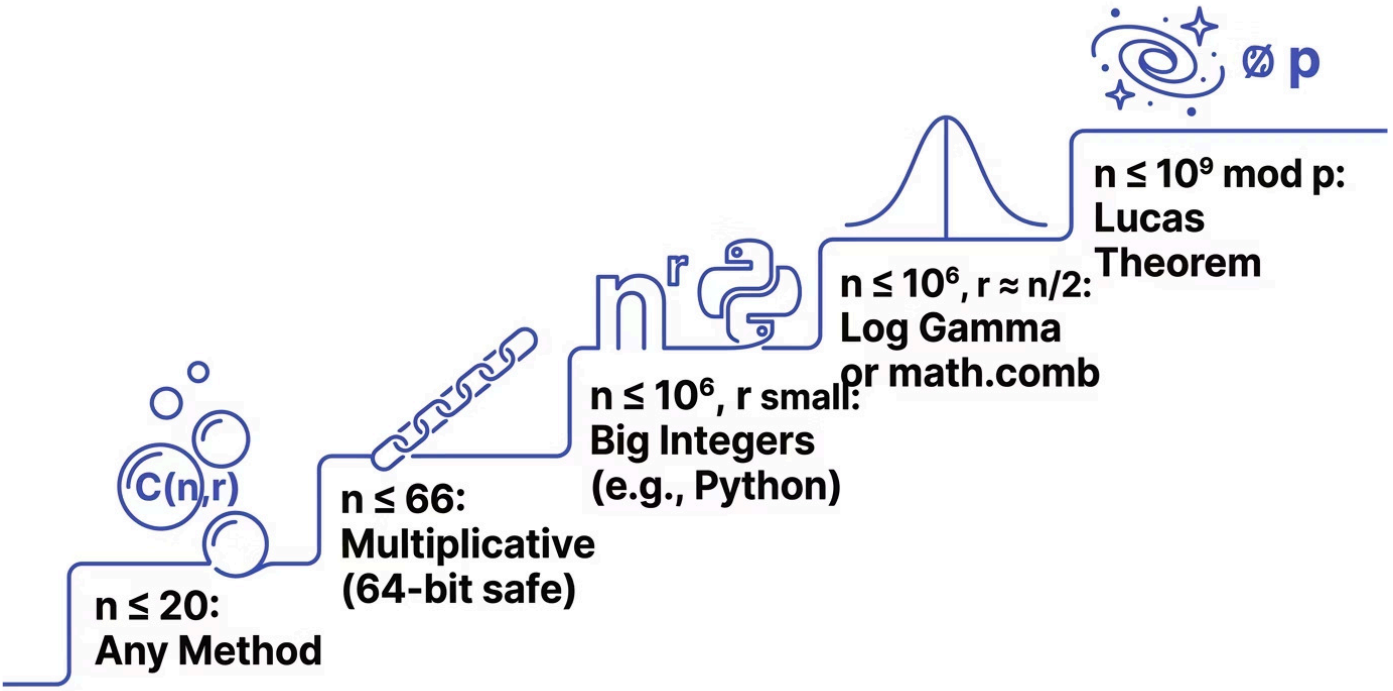
Modular Inverse for Mod Results

Precompute factorials modulo m and use Fermat's little theorem for the inverse when m is prime.

Lucas Theorem for Huge n

For n up to 10^9 modulo prime p, decompose n and r in base p. Time: O(log_p n).

Scalability Strategy by Input Size



Selecting the appropriate scaling strategy is critical in production systems. The staircase above illustrates how algorithmic complexity escalates with input size — each tier demands a fundamentally different approach. Enterprise systems handling combinatorial queries at scale (e.g., genomics pipelines, financial risk engines) must implement tiered dispatch logic that routes requests to the correct algorithm based on n and r.

Design & Architectural Considerations

Key Architecture Decisions

Exact Integer vs. Modular

If exact integer results are required → use `math.comb` or multiplicative with big integers. If only modulo result needed → precompute factorial modulo m plus modular inverses.

Single Query vs. Batch

Single or few queries → multiplicative method O(r). Many queries with same n → precompute all C(n,r) using recurrence for O(1) lookup.

Memory vs. Time Trade-off

Precompute entire Pascal triangle for fast O(1) lookups up to max_n. Compute on the fly using multiplicative for memory-constrained environments.

Design Patterns

Memoisation

Cache factorials, binomial coefficients, and Pascal rows. Avoid recomputation in recursive or iterative loops.

Lazy Initialisation

Build Pascal rows only when needed. Cache computed rows for subsequent queries on the same n.

Strategy Pattern

Implement different nCr strategies (factorial, multiplicative, DP, modular) and dispatch based on input size at runtime.

Security Considerations

⚠ While combinatorics is not a primary security domain, several attack vectors and input validation concerns arise when combinatorial functions are exposed via APIs or user-facing interfaces.

Input Validation

Always enforce: **0 ≤ r ≤ n** and both values non-negative. Return 0 or raise a descriptive error for invalid inputs. Never silently produce incorrect results.

Denial of Service

Attackers may request $C(10^9, 5\times10^8)$ via a public API, triggering enormous computation. Mitigation: enforce upper bounds on n (e.g., $n \le 10^6$) and implement rate limiting on combinatorial endpoints.

Big Integer Memory Exhaustion

$C(10^6, 5\times10^5)$ yields a result with approximately 300,000 decimal digits. Returning this as a string could exhaust server memory. Cap maximum n and return modular results for large inputs.

Side-Channel Risks

In cryptographic contexts where binomial distributions are used for blinding, ensure constant-time algorithms. Variable-time combinatorial computations may leak information through timing analysis.

✓ **Best practice:** Use Python's `math.comb` which includes built-in input validation, handles edge cases correctly, and is implemented in optimised C with no unbounded computation risk for reasonable inputs.

Best Practices

✅ Do's

Area	Recommendation
Design	Use symmetry: compute `min(r, n−r)` to reduce loop iterations by up to 50%
Development	Prefer `math.comb` or `scipy.special.comb` in production over custom implementations
Security	Validate inputs: enforce $0 \le r \le n$ and limit n to a reasonable maximum (e.g., 10^6)
Performance	For repeated queries with same n, cache factorials or use Pascal DP for O(1) lookup
Operations	Log extremely large requests to detect abuse and monitor for anomalous computation patterns

❌ Don'ts

Area	Avoid
Design	Do not compute full factorials for large n — use the multiplicative method instead
Development	Do not use recursion for factorial beyond n=1000 — risk of stack overflow
Security	Do not expose combinatorial functions to high-frequency anonymous requests without rate limiting
Performance	Do not recompute nCr in a tight loop without caching — use memoisation or precomputation

Common Mistakes & Pitfalls

🟢 Beginner Level

Mistake	Why It Happens	Impact	Correct Approach
Off-by-one: C(n,0) = 1, C(n,1) = n	Misunderstanding base cases	Wrong result	Memorise base cases explicitly
Assuming 0! = 0	Thinking 0 has no product	Cascading errors	0! = 1 by definition (empty product)
Overflow with 64-bit integers	n=30, r=15 already exceeds 2^{31}	Silent wrap-around	Use Python ints or check before computation

🟡 Intermediate Level

Mistake	Why It Happens	Impact	Correct Approach
Integer division without ensuring exact divisibility	Naïve loop order	Wrong result (floor)	Multiply then divide interleaved: `res = res * (n−r+i) // i`
Updating Pascal row in forward order	Naïve loop direction	Incorrect row values	Iterate in reverse when updating 1D array
Forgetting C(n,r) = C(n,n−r) symmetry	Not thinking about optimisation	Unnecessary computation	Always set `r = min(r, n−r)`

🔴 Advanced / Production Level

Mistake	Why It Happens	Impact	Correct Approach
Using float gamma for exact integer results	Performance optimisation	Rounding errors	Use integer arithmetic for exactness
Recursive factorial for n=100,000	Misuse of recursion	Stack overflow	Use iterative loop always
Modular division without modular inverse	Assumes normal division works in modulo	Wrong modular result	Precompute inverses using Fermat's little theorem
Applying Lucas theorem for non-prime modulus	Generalisation error	Wrong result	Lucas theorem is valid only for prime moduluss

nCr vs. nPr: Combinations vs. Permutations

Feature Comparison

Feature	nCr (Combinations)	nPr (Permutations)
Formula	n! / (r! (n−r)!)	n! / (n−r)!
Order matters?	No	Yes
Growth rate	Slower	Faster (larger values)
Use case	Lottery, subsets, committees	Rankings, scheduling, passwords
Relationship	nCr = nPr / r!	nPr = nCr × r!

Decision Tree: Which nCr Method?

Need Exact Integer?

No → Use log-gamma approximation (float). Yes → proceed to next decision.

n Small (< 10^4) and Many Queries?

Yes → Precompute Pascal triangle DP for O(1) lookup. No → proceed.

Single or Few Queries?

Yes → Use multiplicative method O(min(r, n−r)). No → consider precomputed factorials with modular inverse.

Learning Summary & Revision Cheat Sheet

Core Formulas

Concept	Formula / Rule
Factorial	$n! = n \times (n-1)!$, $0! = 1$
Binomial coefficient	$C(n,r) = n! / (r!(n-r)!)$
Symmetry	$C(n,r) = C(n, n-r)$
Pascal's rule	$C(n,r) = C(n-1,r-1) + C(n-1,r)$
Binomial theorem	$(x+y)^n = \Sigma\ C(n,r)\ x^{n-r}\ y^r$
Multiplicative form	$C(n,r) = \Pi_{i=1}^{r} (n-r+i)/i$
Lucas theorem	$C(n,r) \bmod p = \Pi\ C(n_i,r_i) \bmod p$
Row sum	Sum of row n in Pascal's triangle = 2^n

Key Takeaways

Factorials Grow Explosively

$20! \approx 2.4\times10^{18}$ fits 64-bit. 21! overflows. Always use big integers for $n \geq 21$.

nCr Counts Unordered Subsets

Fundamental in probability, statistics, genetics, and combinatorial algorithms.

Pascal's Triangle Uses Only Addition

No multiplication or division required — making it numerically stable and overflow-resistant.

Multiplicative Formula is Optimal

Best for single large-n queries. Symmetry halves computation. Always set $r = \min(r, n-r)$.

Common Interview & Certification Questions

1

Compute C(10,3) Without a Calculator

Answer: C(10,3) = (10×9×8)/(3×2×1) = 720/6 = **120**. Also: how many committees of 3 from 10 people? Same answer.

2

Prove Pascal's Rule Combinatorially

Choose a distinguished element x from n items. Either x is included in the r-subset (choose remaining r−1 from n−1) or excluded (choose all r from n−1). Hence C(n,r) = C(n−1,r−1) + C(n−1,r).

3

Sum of Row n in Pascal's Triangle

Answer: $\mathbf{2^n}$. Setting x=y=1 in the binomial theorem: $(1+1)^n = \Sigma\, C(n,r) = 2^n$.

4

Which is Larger: C(100,50) or 2^{100}?

$C(100,50) \approx 1\times10^{29}$. $2^{100} \approx 1.27\times10^{30}$. Therefore $\mathbf{2^{100}}$ **is larger** — it equals the sum of all entries in row 100, of which C(100,50) is just one (albeit the largest).

5

Why Does Multiplicative Method Yield Exact Integers?

After each step i, the running product equals C(n−r+i, i) — always an integer. The intermediate result is always exactly divisible by the running denominator product, guaranteeing integer arithmetic throughout.

6

Time Complexity of Pascal DP for All Rows up to n

Answer: $\mathbf{O(n^2)}$ time and $O(n^2)$ space for the full triangle, or O(n) space using a rolling 1D array.

Recommended Learning Path & Next Topics

Mastery of combinatorics opens the door to a rich ecosystem of advanced topics. Each subsequent area builds directly on the binomial coefficient foundation established here.

Probability Distributions

Binomial and hypergeometric distributions use nCr as their core computation. Essential for statistical inference, A/B testing, and quality control.

Lucas Theorem

Efficient computation of C(n,r) modulo a prime for n up to 10^9. The cornerstone of competitive programming combinatorics.

Catalan Numbers

Defined as C(2n,n)/(n+1), Catalan numbers count valid parenthesisations, binary trees, and polygon triangulations — all rooted in Pascal's triangle.

Generating Functions

Formal power series provide a unified framework for combinatorial sequences. The ordinary generating function for C(n,r) is $(1+x)^n$ — the binomial theorem in disguise.

Dynamic Programming

Many counting problems — grid paths, coin change, subset sum — reduce to combinatorial recurrences structurally identical to Pascal's rule.

DEVELOPED BY TALENCIAGLOBAL

COMBINATORICS SERIES